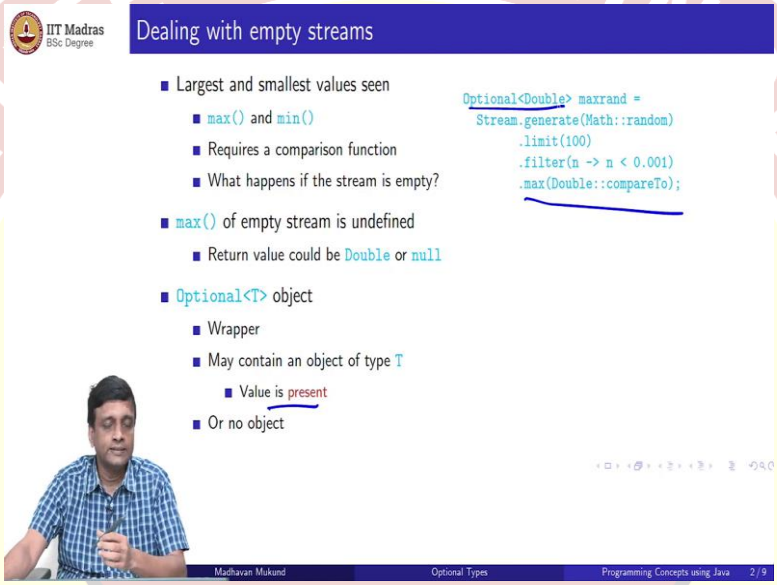
IIT Madras

ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Optional Types

Let us take a look at these optional types which suddenly came up when we were doing stream processing.

(Refer Slide Time: 00:19)



So, the example we looked at were these functions max and min which produce a result from a stream. So, we said that if we were generating a sequence, and then applying some filter to it, which would possibly remove everything from the sequence, we might end up at the end with an empty stream. And if we take an empty stream, and we try to apply a function like max, there is no sensible answer that we can return.

So, this is something that happens quite often that you have some boundary condition in which there is no value that is sensible. So, what do we do in such a situation? So, one conventional way of doing it is to have something which returns a double or a null. Now fortunately, because we are in this object-oriented framework, where null is a valid pointer to an object of any type, this does not break the typing.

So, thing is that I am expecting back something of type double. So, whether you actually give me a pointer to something which is really of type double, or you give me back a value

null, which points to nothing, but that nothing is implicitly of type double, I can check after I call the function whether I got a null or not.

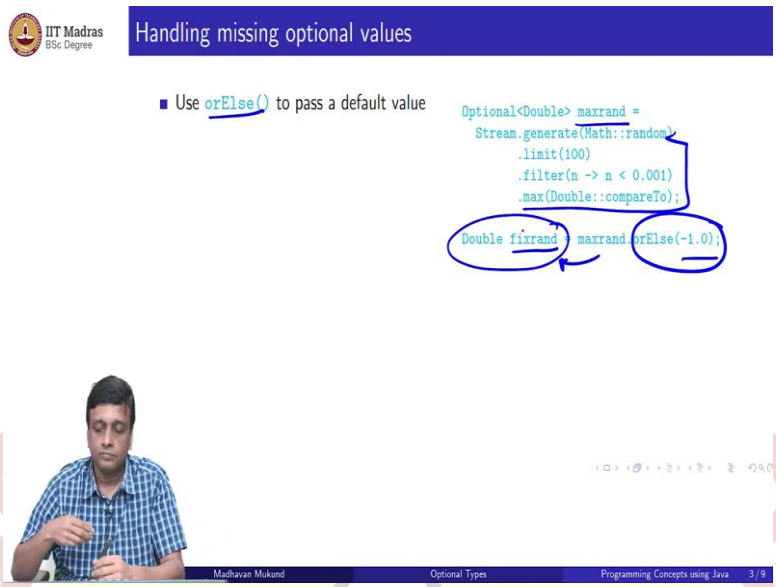
So, this is how, conventionally one works with this possible case, where the function may return a value or it may refuse to return a value because for the argument that you gave it, it could not compute the value. Instead of this, Java has provided this type called optional. So, optional, is a generic type. So, it can work over any underlying type T. So, it wraps up a value of type T, so that you can distinguish between the two cases without getting into this null business.

So, it is in some sense, a nicer or cleaner way to deal with this possibility that the value that you have got, does not really exist. So, when I say that something is of type optional T, there are two possibilities, the default possibility, the normal possibilities, and there is actually a value of type T. In this case, we say that the value is present. But it is also possible that there is no value of type T, so it is an empty box.

Now, that empty box is different from a null pointer, it is not a null pointer, it is an empty box. So, we say that it is empty. So, there is no object. So, either there is an object or there is no object. So, what we will see now is how to deal with these things. So, we have already seen a situation where this happens. So, the max of this stream could generate a double, or it could generate nothing if the stream is empty.

So, this is why it becomes of type optional, double. So, the return value of this whole process is either a double, or it is some empty box. So, we should think of optional as a box. It is like a wrapper. So, it is a box which either contains something of the promise type or it contains nothing, and what we need to do is process this so that we can safely deal with the two situations without getting into looking checking for null and so on.

(Refer Slide Time: 03:17)



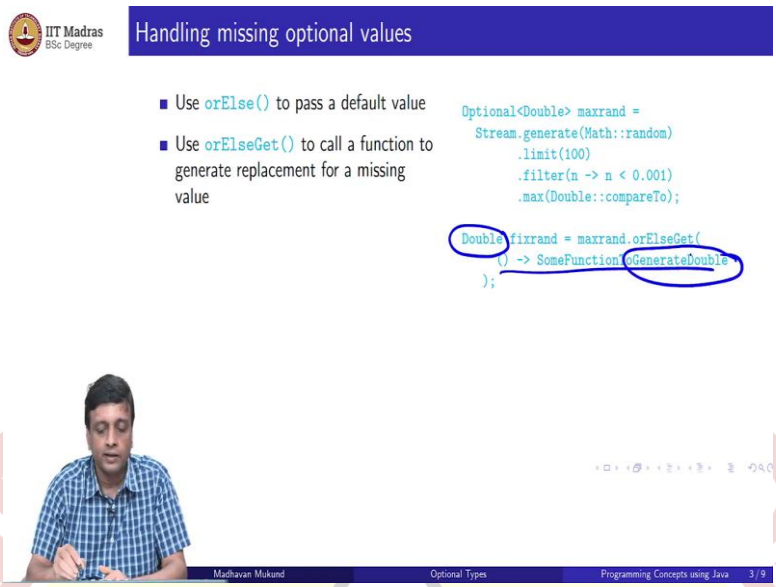
So, the most obvious thing is that you get a box, and if the value is missing, you want to replace it by something, so that somebody else will now realize that something is wrong. So, this for instance, can be done by using orElse. So, I can take an object of type optional, and I can use the orElse and the orElse will trigger if the optional value is empty. So, for instance, here, we take this maxrand to be the maximum of this whole string.

So, now, either this maxrand is a double or it is not a double and I want to convert it to something that is a normal double. So, I want a fixed value of this maxrand. So, fixrand is a real double, but it takes as input an optional double. So, what does it do? It says okay, if it is a real double I will take it otherwise, I will take it to be minus 1.0. Now, we know that this maxrand should be a number between 0 and 1.

So, if I get -1.0, it is clearly not a valid value. So, the important thing is that this triggers only when the value in the type is empty, otherwise, I will just do a normal assignment, I will just take maxrand and pass it to fixrand. So, the default is to pass the value if it is present, and if it is not present orElse tells us how to replace it by some suitable value indicating the lack of presence.

So, I can in this way convert something which has this optional property to something which does not have this option property with some way of indicating that the value was present or not present. So, this part of it is, so this is a more conventional way of thinking about it. If I got it random value which is -1, then I know that it could not have been generated by that. So, therefore, the stream must have been an empty.

(Refer Slide Time: 05:07)

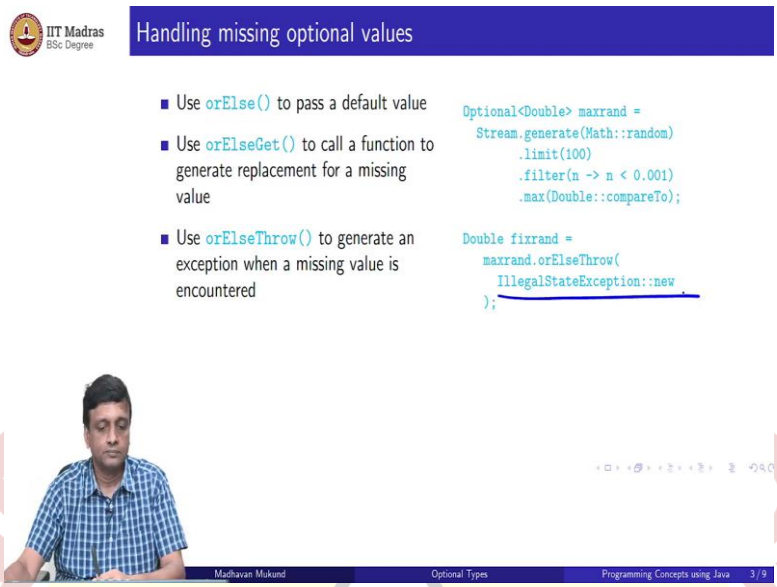


Instead of providing a fixed, so there we provided a fixed value, minus 1. So, providing a fixed value, we can also provide a function that generates a value, but it has to be well typed. Remember that at the end, in this case, whatever you do, you have to produce a double. So, either I get a valid value from maxrand, which is a double, or I provide in the previous case, I provided a kind of signal value like -1 which is double.

Here, I am saying instead of that, you could actually take a function which produces something which is of double, but it could be up to you what you do. So, you might for the first time produce -1, second time, you might produce -2 and so on that, so how many times this thing failed, for example.

So, you can be more flexible than producing a constant value, you can produce values which are dependent on some other thing, but these must be of the same type. So, I am not given an example, but I have just said some function that generates a double. So, the important thing is, it must generate a double because it must be compatible with the type there.

(Refer Slide Time: 06:06)

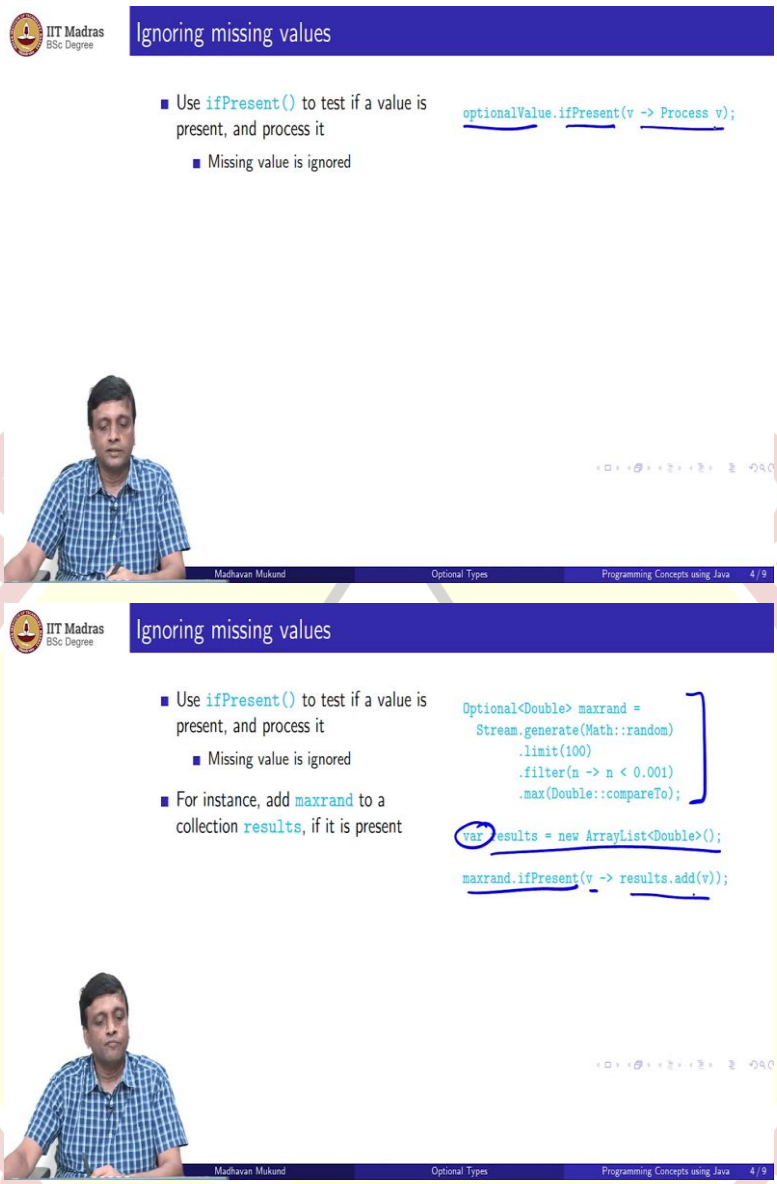


And the last thing that you can do is that if you really do not want these empty types to come, so supposing it is something has gone wrong, and you have no way to fix it. So, remember that, when something goes wrong, and you have no way to fix it, the best way to do it is to announce an exception. So, this is the third option. So, instead of orElse which produces a fixed replacement value, orElse get which generates replacement value from a function, there is an orElseThrow, which actually generates an exception.

Of course, now, if you are generating this exception, you must announce it and all that. But the thing is that you can say, if I do not find a valid value in the maximum, I will throw an illegal state exception, I will throw a new illegal state exception. So, that is what we are saying. So, there are three things that you can do. So, if you are handling the option value, you can either replace it with a fixed replacement, replace it with a generated replacement, or throw up your hand or throw an exception.

Notice that none of these things did we have to actually explicitly check whether the value is this or that or compare it to anything or something. We use this method which extracted the characteristics, if it was a valid value, that value is present, this method just does not work. It just ignores it. If the value is not present, if it is an empty value, then the method triggers and it does whatever we tell it to. So, this is significantly nicer than saying, if maxrand and equal to null do something, else do something else.

(Refer Slide Time: 07:31)



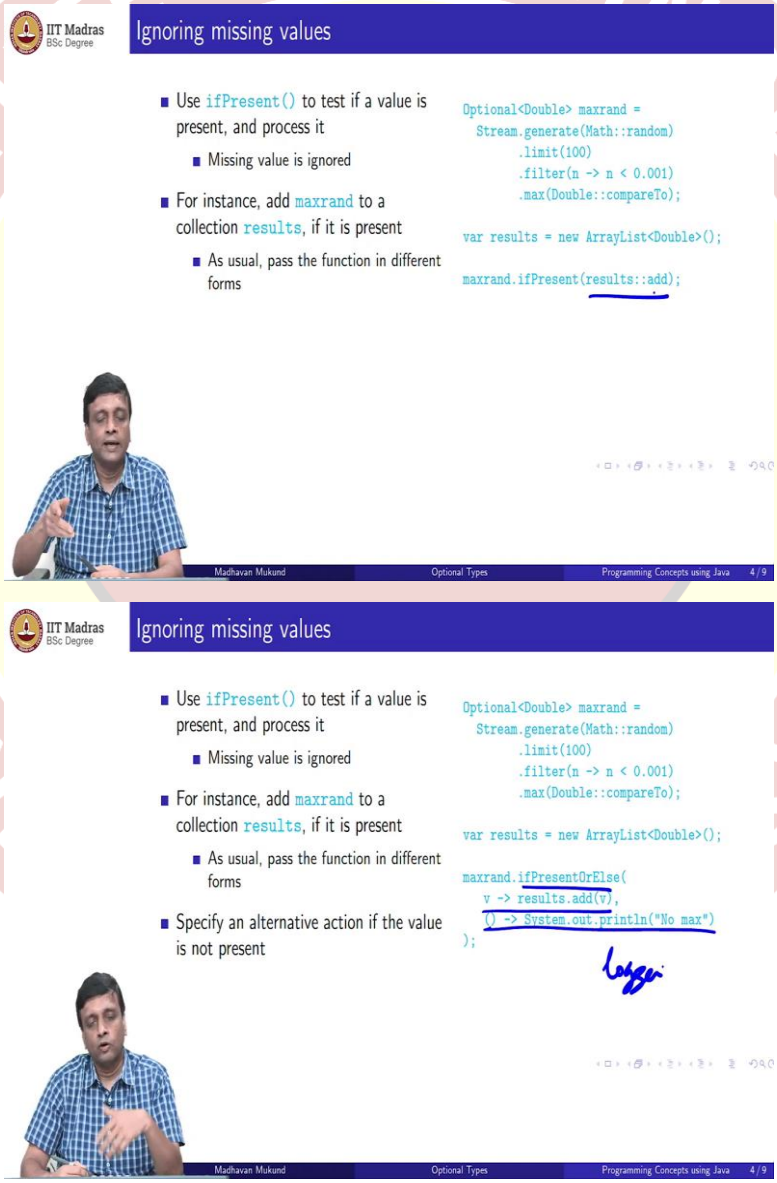
So, sometimes we may not really care. So, if the value is missing, the value is missing, so we will just skip over it. So, there is this other predicate called ifPresent, which essentially triggers. So, in the previous case, the orElse triggered when the value is empty, ifPresent will trigger when the value is present. So, what we are saying is that we have some optional value. And if the value is present, do something with that value. So, we provide a function to do something with that value.

So, for example, supposing we had earlier thing, as before, so we generate the sequence of random numbers, and we filter out from the first 100 of them, we filter out these small ones, and then we want the maximum. Now, we want to do this a number of time maybe or in across various things and store the results in some ArrayList. So, notice here that we are

using this var, implicit type inference kind of situation, because this is going to be an ArrayList, because of the right-hand side.

So, now what I want to say is that if the value is generated properly, then store it in this added to this ArrayList. Otherwise, just do not bother. So, this will say, if the maxrand value is present, then take v and add it to results. If the value is not present, then gracefully ignore it, do not throw an exception, do not kick up a fuss, do not throw any errors and so on, that is what this ifPresent is doing.

(Refer Slide Time: 09:05)



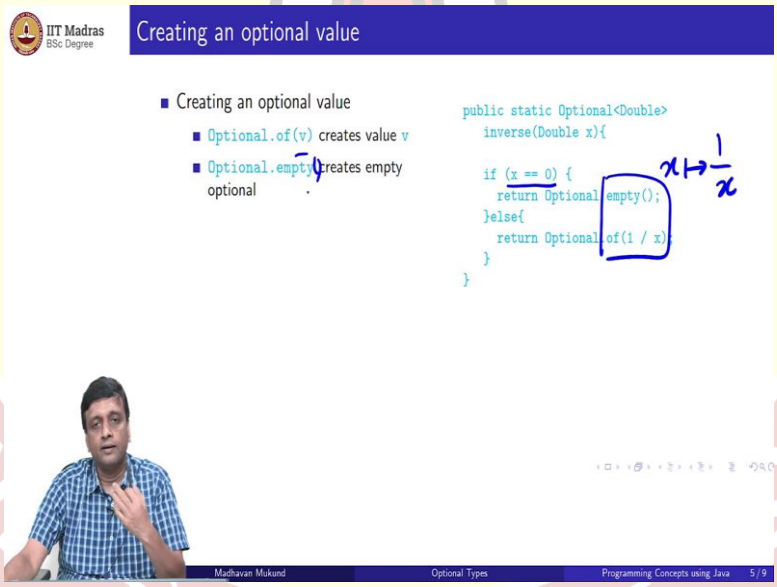
And of course, we could equivalently write that by saying use the add function from results and apply it to results value. So, remember, these are various ways of calling Lambda expressions, you can either give the expression or you can call a static value for a class or

you can call method which is associated with a class or with an object. So, in this case, we are saying apply this method to this object.

We can also give an else, this is saying what to do if the value is present. So, what to do with the value is not present can also be provided. So, we can have ifPresent, now we can have ifPresent orElse, so we have a different method, which does two things. So, it takes one function. The first one is the if part, if the value is present, we add it to results as before.

If the value is not present, then in this case, maybe we just print out a diagnostic, we could also put it into a logger or something. Remember we talked about logger, so printing out will happen at the time when we are running it, but logger is a little bit better, because then you can examine it later on. So, we could log there. So, there are two functions now. The first argument is the if, second part is the else.

(Refer Slide Time: 10:15)



So, that is as far as consuming. So, somebody is producing optional values and giving it to us. In this case, in the previous case, it was generated by max function, but somebody else had written the max function, max function was not something that we wrote. So, the option value is coming from somewhere.

And what we are looking at is how to take this option value and do something sensible to it, how to extract the non-empty part or how to replace the empty part by something else, or how to take throw an exception and so on. Now, what if we want to generate an optional

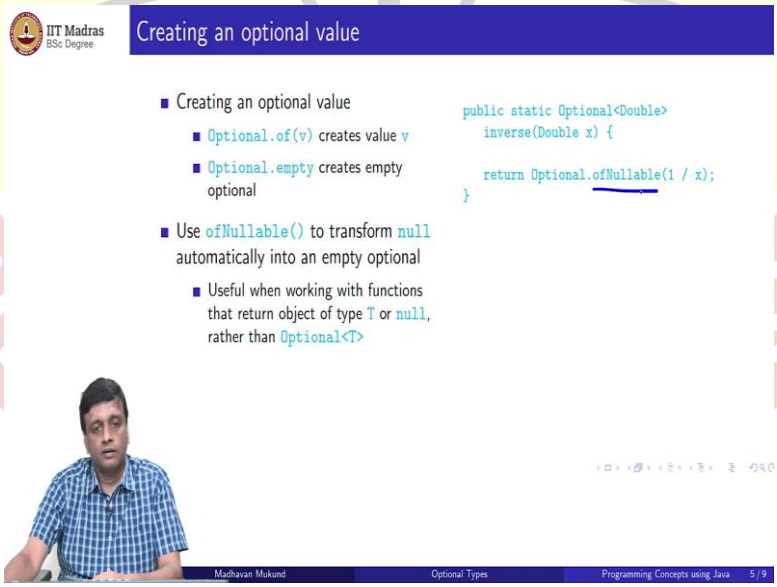
value. So, here is a typical situation you are computing some arithmetic expression, and this arithmetic expression may not have a valid value. So, here we are taking inverse 1/x.

So, we are taking x and we are mapping x to 1/x. Now the problem, of course, is that if the argument given to us is 0, then 1/x is not defined. So, what we want to say is that this function will normally take a double and return a double, but optionally, it may return no value. So, here, what we can say is that if the input is x is 0, then we return and this is now the, the part that is new.

So, instead of just returning 1/x, we return an optional dot empty in case the x is 0, and an optional, we create a double if, as the value part of this optional, so remember, we have to kind of put it into an optional box. So, this optional dot of(1/x) will take this 1/x and put it into an optional box. So, optional.of(v) creates a value or an optional value, where v is present as the value and optional.empty, creates an empty optional.

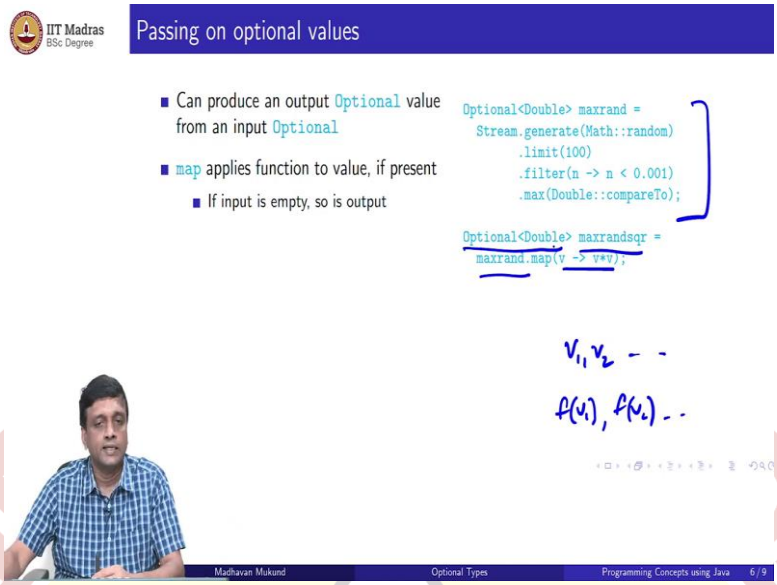
So, this is the basic way of creating an optional value. Now, this is a very common way of doing it, which is that you want to do something and then it is kind of the value that you want to pass is null and so on.

(Refer Slide Time: 12:17)



So, you can actually do it directly. That if then else case, by saying that if 1/x kind of gives you a sensible value, then create that value, otherwise create an empty. So, ofNullable transforms a null or a value that you do not, is not defined into something of the right. So, this is a special case of this. So, we will see where to use this.

(Refer Slide Time: 12:45)



So, now, what we earlier saw was how to take an optional value and convert it to a regular value, we wanted to take a maxrand which is of optional double and produce a real fixed rand which is of type double. But sometimes we may want to just propagate, we might want to take an optional and produce an optional. So, we just want to keep passing this optional. And at the at the end, somebody is going to extract it out. So, this is optional, is like in some in between state.

We do not know, at any point the value may have disappeared, it might have become empty. And finally, somebody is going to resolve it by extracting the value from it. So, a function like map. So, remember what map does. So, map for a stream takes a stream of values and applies each element one at a time, and produces output which is the function applied.

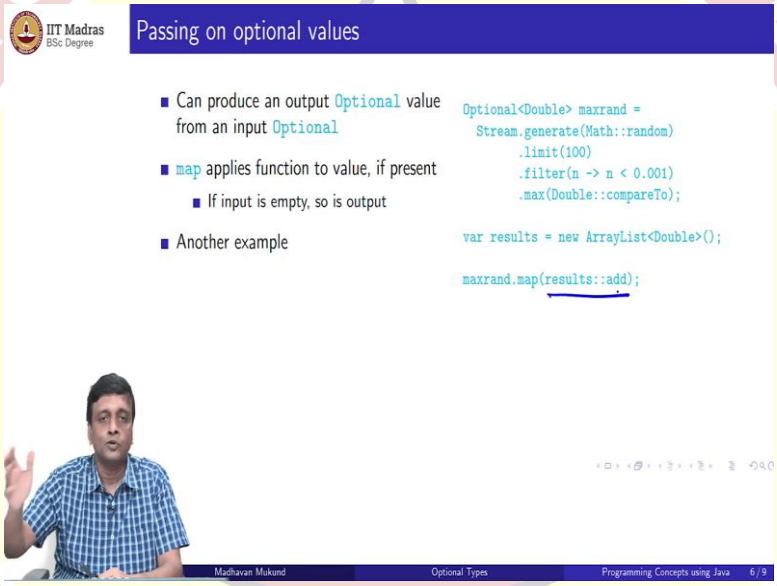
So, if I give you a stream of values v1, v2 and so on, and I give you a function, it is going to produce an output which f(v1), f(v2) and so on. So, this is what matters. Now, what we can think of in terms of optional values is that it is a stream of size 0 or 1, either there is a value or there is no value. And now map turns out to be in this context for this optional type.

So, it is a different map, because it is not the map of stream, but the map associated with the optional type. This map is kind of robust to this 0 1 case. So, if the value is there, it will apply the map, if the value is not there, it will just quietly ignore it. So, here for instance, supposing we want to square this maximum random value. So, as usual, the first part is the same, we are going to find the largest random number out of the first 100 which meets this very selective small random number criterion.

And now what we say is that I want to take this maxrand and I want to convert it into v* v, I want to take v and square it. So, I will apply this map. Now, there are two possibilities I actually got a maxrand in which case I will this maxrand square will now be the square of that, or I did not get anything.

The maxrand was empty because the stream was empty and now map will produce an optional double whose value is empty or not value empty which is empty. So, this is the thing. So, it produces, takes an optional produces an optional, if there is a value, it automatically applies the function produces a value. If the value is not present, it takes an empty produce an empty. So, that is the nice thing about this map.

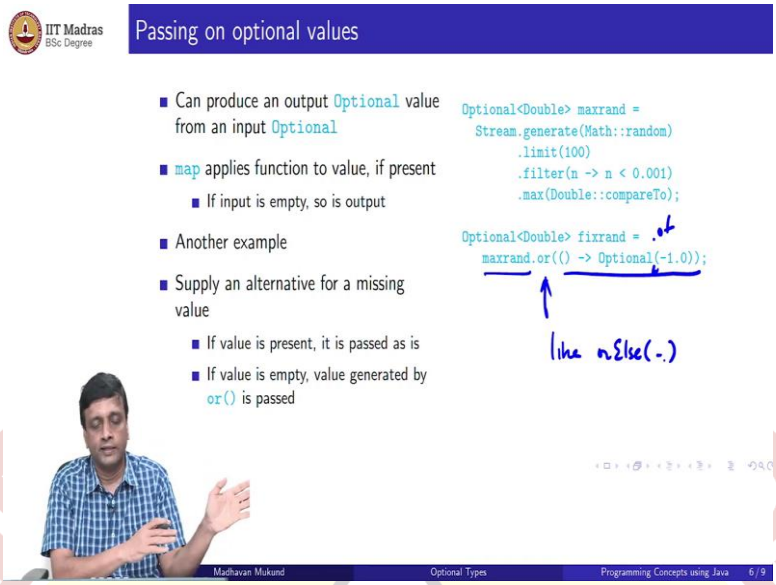
(Refer Slide Time: 15:15)



So, you can do more than that you do not have to actually produce an output in that particular sense, you can do results::add as we saw before. So, we can say that if it is not empty, then put into results. If it is empty, just ignore it and proceed. So, therefore, all the ones which are actually there will get mapped.

So, map is robust in in that sense that it will ignore the empty, so either it will produce an empty object, if you are producing an output. Or if you are applying it to do something else, then it will do something else only if the value is present.

(Refer Slide Time: 15:44)

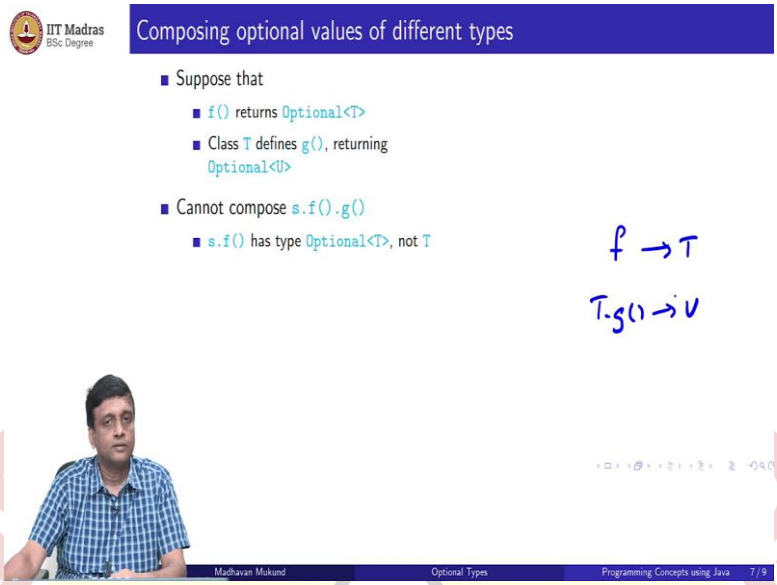


So, like we saw before, you could also say, so this is an optional thing. So, you can also produce a new value in place of an empty value. So, for instance, we said before that we had this orElse, so the orElse was to convert an optional double into a double. So, it said either take the real value and make it a double, or take the empty value and make it -1.

Now here, what we are seeing is you take an optional value, and we convert it to an optional double, either we copy the value or we apply this function. So, what does this function do, it produces an optional of value -1.0. So, this should probably be dot of, if I should change it. So, it producer an option. So, remember how to produce an option. You say, optional dot of and give a value.

So, this will produce an optional value of -0.1. So, the output is now going to be something which has a value present, either it is going to be the value that I got from maxrand, the or will trigger so this is like orElse etcetera. So, this will trigger only if the value is empty. And if the value is empty, it will produce an output whose value is not empty, but will rather be the outcome of this function. So, it will basically replace empties by something new and replace non empties by whatever they had before.

(Refer Slide Time: 17:12)

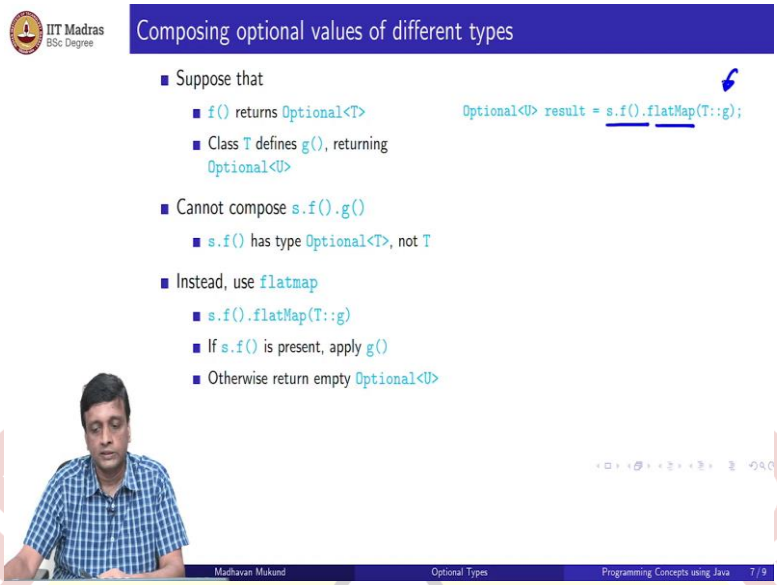


So, normally, when we have functions which produce values of compatible types, we can apply them one after the other. So, if function f produces an output of type T, and there is something which takes an input of type T and produces a type U, you can take f and compose it with that other function. But what happens if we have optional, so supposing f returns an object, which is not of type T, but is of type optional T.

So, it could if it is there, it is a T, but it may not be there. And supposing in T, we have a function g(). So, if we get an actual object of type T, we can always apply g() to it. And this g() produces, again, something of type U, but not necessarily of type U, something optional type U. I want to know compose this, I want to apply g() to these outputs of f().

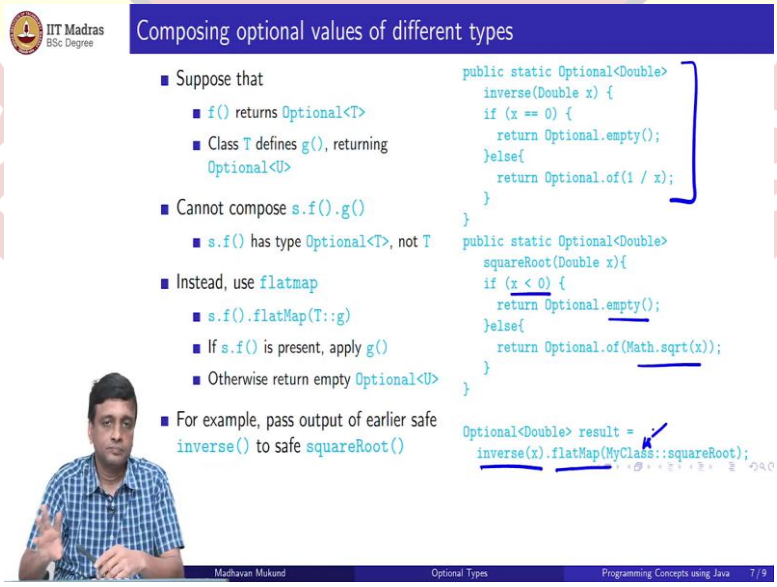
So, if I write s.f(), so supposing if I apply f to an object s, so I have an object s, whatever it is, when I run the method f() on it, it produces a T and on that T, I can produce a g(), so this would normally be allowed. So, if f() produced a T. and if T dot g() produced a U, this would work. But the problem here is that s.f() does not produce a T, it produces an optional T. So, then that g may not make sense. So, I cannot just write this.

(Refer Slide Time: 18:33)



So, the solution to this is to use something called a flatmap. So, what you do is you tell a s.f() to pass it not to g() directly, but to flatmap. And what flatmap does is it takes this argument, the function that is to be used on its input. So, remember, what map did. So, map took its input. And if it was a valid number, or a valid value, it applied function, if it was not a valid value, it just passed it as empty. So, here flatmap is doing the same thing. So, it is taking its input, and it is applying a function g() to it, but it is being used in this context, where the input is coming from another function.

(Refer Slide Time: 19:13)



So, for example, we looked at the safe inverse. So, the safe inverse, generated an optional value of 1/x if the value of 1/x was valid, otherwise, it is generated empty. And now, what

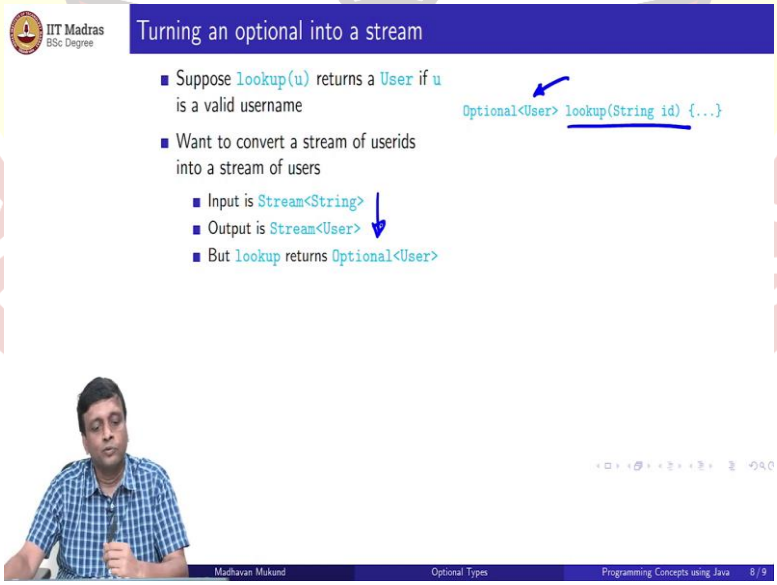
we can say is that we have a safe square root. So, what does safe square root says? It says that if the value is negative, then generate an empty because I cannot take the square root of a negative number.

Otherwise, just apply the normal math square root function to it. So, now, what I want to do is I want to take the output of the safe inverse and pass it to the safe square root. Now normally, this should be allowed because the output of the safe inverse is going to be a double and that double can be passed to square root. But because I cannot assume that the output of the inverse will be correct, it is going to be optional double.

So, now instead, what I will do is I will say inverse of x and I will compose it with not directly the square root, but flatmap, I have to tell it where it is. So, I am just assuming this whole thing written in some new class, which I have defined called MyClass. So, I am looking for the square root function in MyClass.

So, this flatmap, we will use the flatmap a little bit. So, let us just try to understand this. So, what flatmap is saying is that I compose two functions where the first function may produce an optional output, which is compatible with the input of the second function, then I use flatmap.

(Refer Slide Time: 20:34)



So, now let us look at the question of turning an optional into a string. So, let us look at it through an example. Supposing I have a function, which will look up some database of persons, some users. So, this should be maybe. So, this is a lookup view, which returns a

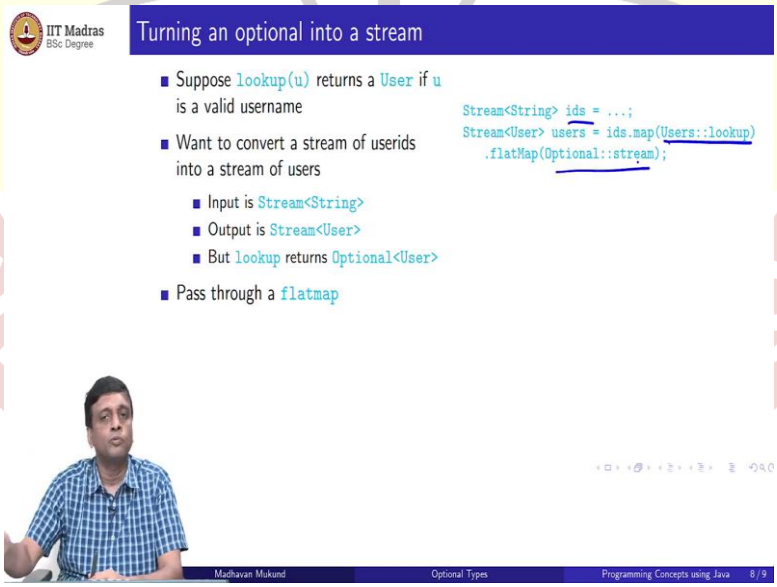
user if u is a valid name. So, u is some kind of user name, maybe it is somebody's login ID on a system or something.

And if it is a valid login ID, so it says a string. If this is a valid singer, it returns an object describing that user. So, now what I want to do is I want to take the stream of these user IDs and convert it to a stream of users. And how will I do that? For each input string that I get in my stream, I will look it up. And if it is a valid user, I will produce it, if it is not a valid user, I will not produce it.

So, this is what lookup is doing. The lookup is producing an optional of a user. So, if it is a valid user ID, it will produce a valid user object. If it is not a valid user ID, it will produce an empty object. That is what this option does. So, the problem with this is I cannot just directly use a map. I cannot use a map here because I have something which is coming in.

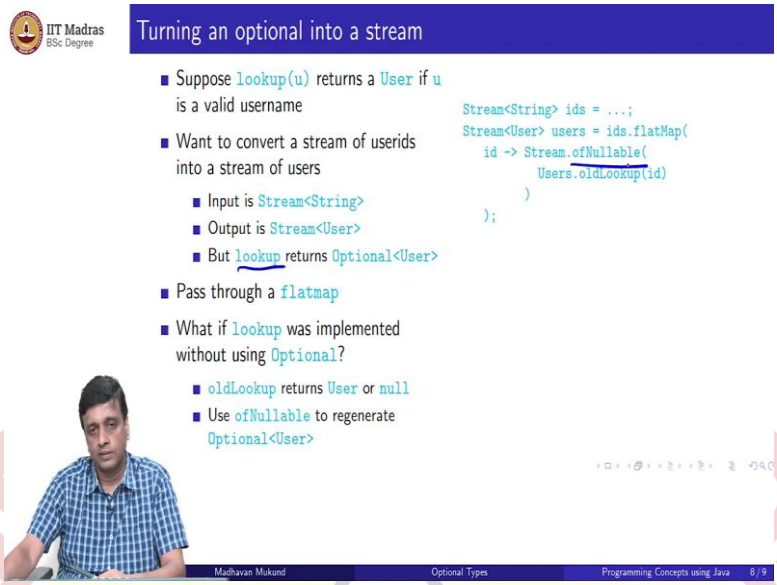
So, this is the same problem that we said before. I have an output, which is sort of compatible with my next step. But it is not because in between, I have this option. So, I have to take this optional and do something to it. So, as you would expect, you would use a flatmap.

(Refer Slide Time: 22:19)



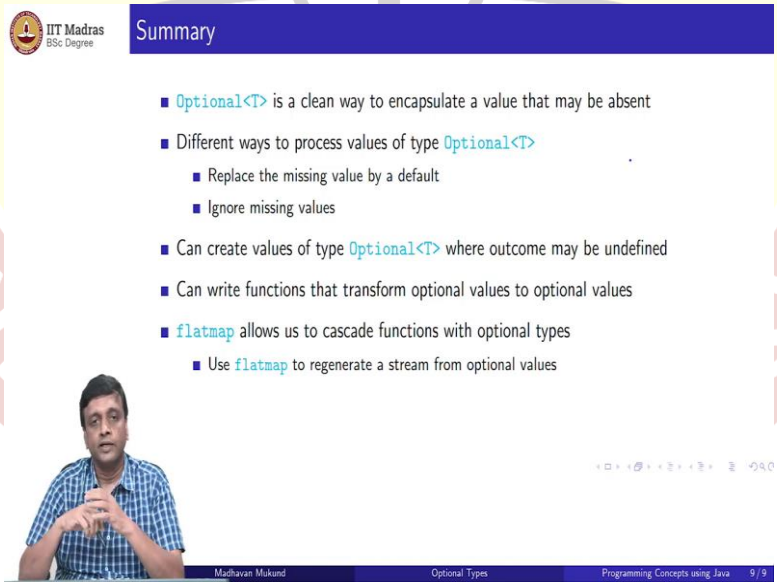
So, you will pass this instead through a flatmap. So, what we will do is that supposing I have this stream of ids, which is of type string. Then I will first map it using this lookup function. So, when I map it using this lookup function, what it will do is it will produce a set of things which are of type maybe, this whatever optional user, and now I flatmap it and I produce a stream, so each of these will produce a stream out of it.

(Refer Slide Time: 22:51)



So, what if this lookup was actually an old-style lookup, which produced either a user or a null, then I can use this version. This stream.ofNullable, which will now take me from a valid value to a valid value, and from a null value to an empty value. So, this is where this ofNullable comes again.

(Refer Slide Time: 23:18)



So, optional T is a fairly interesting way of dealing with values which may or may not be defined. So, instead of having to distinguish between a null and a valid value, it allows us to encapsulate this into a box, which either has a value present or is empty. And then we can use functions which are defined on this box.

So, this is really an object-oriented way of doing it. We do not need to know, for example, how the empty values represented, we never look inside and say, what does it mean for this value to be absent, is it null, is it something else that we do not know. What we know that we have functions like that orElse and so on, which will allow us to do things like replace a missing value by default value if the value is missing, without knowing why it is missing the sense.

We do not know what is the representation of the missing value, we can just say either give me the value that is present or replace it by this and it will be done automatically or we can say ignore it, if the value is present, do something otherwise do nothing. So, we can ourselves create these values we said.

So, we can write functions, which when they are not able to produce a valid answer, instead of producing some nonsensical number or some null or something, we can actually produce an optional type as an output, then we can write functions which take these optional types and produce new optional types. And we can cascade these things using flatmap.

So, these optional types come naturally in stream processing, because of the possibility that some of the things that we are trying to do on streams will die out because the stream becomes empty. So, this is something but it also applies as we saw when we do this safe arithmetic function like safe square root and safe inverse and so on.

So, it is quite often that we need to write something but of course now the problem is that if we write a function which produces optional types, then people who use this function need to know how to deal with optional types. So, that requires a little bit of cooperation on their part.

So, you have to assume that the person who uses your function knows that optional types work in a particular way, but modular that this is a nicer way of dealing with these functions where values could be undefined, rather than assuming that the undefined value is mapped to null or something specific like that.